

CS5787: Exercises 3

Python Notebooks and READMEs for practical section:

https://drive.google.com/drive/folders/1PiwLXKC7ka980t9A1h02U0HNGP_wqSOR

Avinash Nair
an586

Jasper Levy
jl4537

1 Theory: Question 1 [40 pts]

a. In the case where $\theta \neq 0$, the KL divergence is infinite, as there is no overlap between P and Q . This causes the log term in the KL divergence formula to be undefined, leading to infinite divergence.

The Jensen-Shannon (JS) divergence is given by the following equation:

$$D_{JS}(P||Q) = \frac{1}{2}D_{KL}\left(P||\frac{P+Q}{2}\right) + \frac{1}{2}D_{KL}\left(Q||\frac{P+Q}{2}\right)$$

Where $D_{KL}\left(P||\frac{P+Q}{2}\right)$ represents the KL divergence between distribution P and the average distribution $\frac{P+Q}{2}$. Similarly, $D_{KL}\left(Q||\frac{P+Q}{2}\right)$ represents the KL divergence between distribution Q and the same average.

There is no overlap in distributions P and Q 's x dimension, but both of these KL divergence terms are finite. The average distribution $\frac{P+Q}{2}$ has mass at both $x = 0$ and $x = \theta$, avoiding the infinite penalty that occurs in KL divergence when the supports are disjoint. This results in the JS divergence being finite, and for symmetric distributions like P and Q , it simplifies to $\log(2)$.

Finally, the Wasserstein distance measures the cost of moving probability mass between P and Q . The minimal cost to move all the mass from $x = 0$ in P to $x = \theta$ in Q is proportional to θ , giving a Wasserstein distance of $|\theta - 0| = \theta$.

b. In the case where $\theta = 0$, the KL divergence will be 0, as the distributions are identical. The JS divergence will be 0 due to identical distributions. The Wasserstein distance is also 0, as it takes no movement to get from distribution P to Q or vice versa.

c. The Wasserstein distance is especially useful in scenarios where the distributions do not overlap. In part a, there was zero overlap, and the KL divergence went to infinity, whereas the Wasserstein distance remained stable and proportional to θ . The Wasserstein distance also works better than JS divergence, as JS also is more concerned with the overlap of distributions, while the Wasserstein distance seeks the actual "distance" between distributions, regardless of overlap.

2 Theory: Question 2 [10 pts]

LSTM: LSTMs have sequential processing, and each token as output depends on the previous state of the hidden state. Each step involves an input-to-hidden and hidden-to-hidden multiplication (one for each of the 4 gates – forget, input, candidate cell state, and output).

Since the dimensions of the hidden state are greater than the dimensions of the input of the model, and

$$d_{\text{input}} \times d_{\text{hidden}} < d_{\text{hidden}} \times d_{\text{hidden}},$$

the term d_{hidden}^2 dominates. Since we perform this operation 4 times (once for each gate), we get:

$$4 \cdot O(d_{\text{hidden}}^2) = O(d_{\text{hidden}}^2).$$

Now, since we do this for each processed token, and we have N tokens, the computational complexity will become:

$$O(N \times d_{\text{hidden}}^2).$$

This accounts for the most complex computation at each sequential step (d_{hidden}^2) for N steps – a sequence of length N being generated.

Transformer:

For Transformers, token generation occurs one token at a time. A key part of transformer model generation is the self-attention component, which means that the model looks at all previously generated tokens to decide which token to generate next as part of the N -long output.

When a token is input into a Transformer, it is converted into a dense vector v_{dense} of a certain fixed dimension. Each token is represented by a dense vector of the same size. Since the generation of each token occurs at each step and is dependent on all previously generated tokens, we have:

$$\begin{aligned} \text{Step 1: } & O(v_{\text{dense}}) \\ \text{Step 2: } & O(2 \cdot v_{\text{dense}}) \quad \text{– to account for steps 1 and 2} \\ \text{Step 3: } & O(3 \cdot v_{\text{dense}}) \\ & \dots \\ \text{Step N: } & O(N \cdot v_{\text{dense}}) \end{aligned}$$

To generate all tokens in the sequence of length N , we need to perform all these steps from 1 to N . This gives:

$$O(1 \cdot v_{\text{dense}}) + O(2 \cdot v_{\text{dense}}) + \dots + O(N \cdot v_{\text{dense}}).$$

The formula for the sum of the first n natural numbers is:

$$\frac{n(n+1)}{2},$$

which is asymptotically equivalent to $O(N^2 \cdot v_{\text{dense}})$ – meaning it behaves similarly to n^2 in terms of growth rate.

3 Practical: Question 3

3. Here is the distribution we trained the GAN on in the playground.

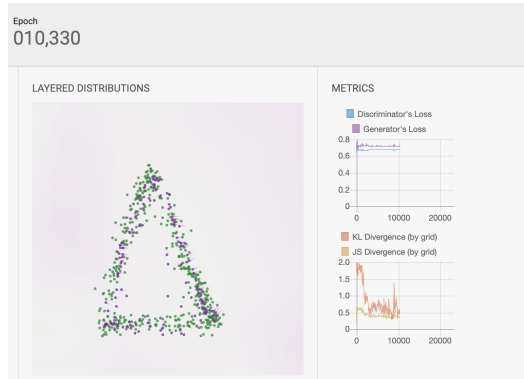


Figure 1: This image captures convergence of the GAN

4 Practical: Question 4 - VAE-M1

We implemented a version of the M1 architecture from “Semi-supervised Learning with Deep Generative Models” (Kingsma et al, 2014) for semi-supervised learning on the Fashion-MNIST dataset by Zalando (2014).

Model/Code Details

Within the VAE itself, we took a modified approach by including convolutional layers. We used a Radial Basis Function (RBF) kernel in our SVM for classification. The readme file provides more information, including how to train and test our model.

Conclusion and Discussion

Our initial approach was to try and follow the paper’s provided architecture as close as possible, but we were getting low accuracies when testing with the SVM. We switched over to include convolutional layers, reasoning that these layers would be able to better capture important spatial information. This improved the quality of the learned latent space.

We also experimented with three kernels in our SVM: linear, poly, and rbf. Each kernel had its advantages, with linear providing a baseline, poly capturing complex relationships, and rbf allowing for non-linear decision boundaries. We found that rbf outperformed the other two kernels, which also provided support for making use of convolutional layers as opposed to strictly linear layers.

Overall, the inclusion of convolutional layers in the VAE, along with using the rbf kernel within the SVM, led to our best classification results.

Readme and visualizations

The accuracies for different numbers of labelled data is available in the readme file. The readme file also additional information for training and testing with saved model weights.

Link to folder

This includes the README for the VAE-M1 as well as the notebook with the code. You can access it [here](#).

5 Practical: Question 5 - GANS and WGAN techniques

We implemented two architectures from the paper “Improved Training of Wasserstein GANs”, by Gulrajani et al (2017). We add to this paper by testing two architectures with three techniques each (DCGAN, WGAN (weight clipping), and WGAN (gradient penalty)). We work with the FashionMNIST dataset to generate images with generator networks after training is complete.

Models

The models that we use in this practical are DCGAN, WGAN (with weight clipping), and the WGAN-GP (with gradient penalty). These models have slightly different hyperparameters that can be explored more in the Gulrajani et al paper (as we used these same settings).

Our Chosen Networks are the following:

Architecture 1:

- **Nonlinearity (G):** ReLU
- **Nonlinearity (D):** LeakyReLU
- **Depth (G):** 4
- **Depth (D):** 4
- **Batch normalization (G):** True
- **Batch normalization (D); layer norm for WGAN-GP:** True

- **Base Filter Count (G):** 64
- **Base Filter Count (D):** 64

Architecture 2:

- **Nonlinearity (G):** ReLU
- **Nonlinearity (D):** LeakyReLU
- **Depth (G):** 4
- **Depth (D):** 4
- **Batch normalization (G):** True
- **Batch normalization (D); layer norm for WGAN-GP:** True
- **Base Filter Count (G):** 32
- **Base Filter Count (D):** 32

For both architectures, the only convergence that failed the first time was for the WGAN-GP model. We had to tweak the beta values and learning rate – aligning it to the paper’s parameters for WGAN-GP – in order to see convergence. Notably, we could not get the generator loss to trend towards 0, and it behaves like a non-decreasing function for both architectures; however, the images generated by the generator were legible and good enough to ensure that the GAN approach was working, and the generator was learning to mimic the train images well.

Conclusion and Discussion

Our architecture 1 produced better images for each model when compared to architecture 2. This is likely the result of the higher capacity to learn detailed features, which is afforded by the larger base count filter for both the generator and the discriminator (64 vs 32). The discriminator is likely a bit better at differentiating real from fake images by capturing richer relationships in the data. In turn, this forces the GAN to perform better as well. The GAN would be provided a better gradient signal in this case, creating better-quality images.

Ultimately, the best performing model and architecture combination is the WGAN-GP (with gradient penalty) on architecture 1 (with a larger base filter count). This generator was able to produce very high-quality images (based on the provided FashionMNIST dataset) in comparison to the other generators that produced fuzzier images.

The WGAN-GP enforces the Lipschitz constraint with the gradient penalty during training. This ensures a smooth gradient flow and therefore improved training. The gradient penalty specifically makes sure that the critic / discriminator doesn’t give erratic gradients, allowing the generator to better learn crisp outputs based on the training data.

Readme and visualizations

The images generated, plotted loss functions, and images generated for two classes of the best architecture (for each model variant) is available in the readme file. The readme file also additional information for training and generating images with saved model weights. See readme [here](#).

Link to folder

This includes the README with plots as well as the notebook with the code. You can access it [here](#).